

# LLM-Augmented IoT Traffic Management System

Zakariya Ba Alawi

Student ID: 220027

Department of Software Engineering

Alfaisal University

Riyadh, Saudi Arabia

Mohammed Bawazir

Student ID: 230035

Department of Software Engineering

Alfaisal University

Riyadh, Saudi Arabia

Ahmed Bin Halabi

Student ID: 220026

Department of Software Engineering

Alfaisal University

Riyadh, Saudi Arabia

Saad Alkeridis

Student ID: 220621

Department of Software Engineering

Alfaisal University

Riyadh, Saudi Arabia

Mohmamed Haythem

Student ID: 220601

Department of Software Engineering

Alfaisal University

Riyadh, Saudi Arabia

Abdulrahman Hisham

Student ID: 220335

Department of Software Engineering

Alfaisal University

Riyadh, Saudi Arabia

**Abstract**—Adaptive embedded traffic controllers are now widely deployed at small intersections, but the resulting systems are opaque to non-technical operators who cannot inspect, override, or interrogate them without specialist tooling. We present an LLM-augmented overlay for an ESP32-based two-direction traffic intersection that gives operators a single natural-language interface for both control and reporting. The hardware layer (ESP32-WROOM-32S, four HC-SR04 ultrasonic queue sensors, six signal LEDs) runs its existing adaptive timing algorithm unmodified. A FastAPI bridge subscribes to the existing MQTT telemetry topics, maintains a thread-safe rolling state window of about sixty seconds, and exposes a chat endpoint that calls the Anthropic Claude API using a typed tool-use schema. Two tools, `traffic_override` and `emergency_priority`, constrain every command the model can emit; the firmware adds one new subscription (`iot/traffic/override`) and applies bounded-duration overrides that fall back to the adaptive algorithm. We evaluate the system on a labelled dataset of forty operator utterances spanning four categories (command, ambiguous, out-of-scope, reporting) against a deterministic regex baseline. The bridge reaches 97.5% overall accuracy versus 75.0% for the baseline, with tool-call detection F1 of 0.947 against 0.737. All ten LLM-produced tool calls were schema-valid, supporting the claim that constrained tool-use makes the overlay structurally safe by construction. Median latency was approximately 1.1 s with a P90 of 2.3 s. The contribution is a small, reproducible reference design that demonstrates how a typed tool-use interface and an overlay deployment can deliver both safe command translation and grounded conversational reporting on commodity hardware.

**Index Terms**—Large Language Models, Internet of Things, MQTT, ESP32, traffic management, tool use, edge-cloud, natural language interfaces.

## I. INTRODUCTION

Urban traffic congestion is one of the most visible failures of static infrastructure in growing cities, and signalised intersections are the point at which that failure is most often expressed. Adaptive controllers based on inductive loops or, in low-cost installations, ultrasonic queue sensors have reduced average delay relative to fixed-time control [1], [2]. Their effectiveness depends on a control engineer being able to tune, monitor, and override them, and that dependency does not survive contact

with the operational reality of small municipalities, university campuses, or industrial sites, where the people who watch traffic on a day-to-day basis are dispatchers, security staff, or facility managers rather than control engineers. The interface that those operators have to work with is typically a numerical dashboard or, worse, no interface at all. When traffic backs up unexpectedly the operator has no usable mechanism to intervene other than calling someone who does.

This gap between an increasingly adaptive control layer and an unchanged human-facing layer is the motivating problem of our project. Recent work on Large Language Models has shown that natural-language interfaces can be applied effectively to structured technical systems [4], [5], including, in early studies, to traffic-related reasoning [6]–[8]. The same studies make clear that letting an LLM emit free-form text into a control bus is not safe: model output drift, hallucinated parameters, and malformed payloads are routine failure modes that any production system has to engineer around. The constrained tool-use interfaces now exposed by major providers [9], [10] offer a clean answer: the model produces structured calls against a typed schema, the runtime parses them with no string surgery, and the set of possible commands is finite by construction.

We use this design point to build a working LLM-augmented overlay for an existing ESP32/MQTT intersection. The system has three properties that together separate it from prior work on LLMs in traffic. First, it is an overlay rather than a replacement: the firmware adaptive timing algorithm is untouched, and the LLM communicates with it only through one new MQTT topic and a bounded-duration override convention. Second, the operator interface combines command translation and conversational reporting in a single chat surface, so a question like “which direction is busier?” and a command like “give the busier direction priority for ten seconds” both route through the same endpoint and share the same rolling state context. Third, the evaluation uses a labelled four-category dataset and a deterministic regex baseline that any subsequent overlay can replicate, which lets us report not just accuracy

but also structural validity and refusal behaviour on ambiguous and out-of-scope inputs.

The remainder of the paper is organised as follows. Section III surveys related work on LLM–IoT integration, natural-language interfaces for embedded control, and MQTT-based traffic systems. Section IV presents the five-layer system architecture and explains why tool-use was chosen over prompt-only approaches. Section V documents the firmware override path, the bridge threading model, the tool schema, and the dashboard. Section VI defines the evaluation methodology, dataset, and metrics. Section VII reports results and analyses them by category. Section VIII covers limitations and threats to validity. Section IX concludes.

## II. BACKGROUND

### A. Hardware Platform

The ESP32-WROOM-32S is a dual-core Xtensa LX6 microcontroller with integrated Wi-Fi and Bluetooth, widely used in low-cost IoT projects because of its price-to-capability ratio and the maturity of its Arduino-compatible toolchain. The HC-SR04 ultrasonic distance sensor produces a pulse whose width is proportional to the round-trip time of a 40 kHz sound burst, and is the lowest-cost vehicle-presence sensor in common use for educational traffic systems. We use four HC-SR04 units, arranged as entry and exit pairs on each approach, so that the firmware can maintain a queue counter for each direction by incrementing on entry events and decrementing on exit events. The signal LEDs are driven directly from GPIO with current-limiting resistors.

### B. MQTT and Adaptive Timing

MQTT is a publish–subscribe protocol designed for resource-constrained devices over unreliable networks [16]. Each client publishes to and subscribes to topics on a broker; in this project the broker is HiveMQ’s public test server. Topic strings encode the hierarchical structure of the data; we use `iot/traffic/{WE,EW}/{state,queue}` for telemetry and `iot/traffic/override` for control. The firmware’s adaptive timing algorithm, summarised by Eq. 1, sets each green-phase duration proportional to the queue count at the start of the phase, capped at a hardware-safe maximum. This is the same basic family of algorithm reported in earlier IoT traffic work [1], [3] and is appropriate for a single isolated intersection; multi-intersection coordination is out of scope for the present project.

### C. Large Language Models and Tool Use

Modern instruction-tuned LLMs such as the Claude family used here, the GPT-4 family, and open-weight Llama-3 models can interpret natural-language intent and produce structured outputs against a declared schema [4], [15]. Tool-use APIs are the platform-level mechanism for this: the developer declares a JSON-schema tool, the model may choose to emit a structured call against that tool, and the runtime parses the call directly without string surgery on natural-language output [9], [10]. The key property of this interface for safety-relevant runtimes

is that malformed outputs are rejected by the platform, not by application code, which means the application can rely on the schema as an invariant rather than as a hopeful assumption.

## III. RELATED WORK

### A. Adaptive IoT Traffic Control

IoT-based traffic management has progressed from sensor instrumentation to closed-loop adaptive control over the last decade. Zitouni and Ammar [2] used UPPAAL timed automata to model an MQTT-based intersection and quantified round-trip delays for traffic signal commands over a publish–subscribe bus, establishing MQTT’s suitability for the latency budgets of urban control. Yusoff *et al.* [1] described a smart traffic system using IoT sensors to dynamically adjust phase durations to instantaneous queue length, the same formula family our firmware uses. Pawar *et al.* [3] reported an ESP32 implementation with HC-SR04 ultrasonic sensors and demonstrated that adaptive timing outperformed fixed-time control on a small testbed. These works share a structural limitation: the resulting controllers expose no human-facing interface beyond numeric sensor readouts, and any tuning requires reprogramming the device.

### B. Language Models for Traffic and Mobility

Wang *et al.* [6] introduced Traffic Performance GPT, an LLM agent that interfaces with traffic simulation environments to answer operational questions and propose control actions; under sensor-outage conditions their system reduced delay by 20.4% relative to a non-augmented baseline. Movahedi and Choi [7] surveyed the intersection of LLMs and smart mobility, identifying command translation and conversational reporting as the two interaction modalities with the strongest near-term evidence of utility. Liang *et al.* [8] demonstrated that a tool-augmented LLM can outperform fine-tuned classifiers on traffic incident reasoning without task-specific training, providing direct evidence that constrained tool-use is a viable interface pattern for traffic data. Moraga *et al.* [13] extended the picture to multimodal sensing by coupling UAV monitoring with an LLM-driven SUMO controller. Lai *et al.* [12] examined LLM-driven adaptive signal control in microsimulation and reported that prompted LLM controllers approached learned-policy baselines on average travel time while remaining interpretable.

### C. Natural-Language Interfaces for Embedded Systems

Outside traffic, work on natural-language interfaces to embedded systems has converged on tool-use or function-calling as the safe contract between an LLM and a hardware bus. Le *et al.* [11] described a vision-language framework for IoT querying that uses a typed schema to enforce well-formed outputs. Anthropic and OpenAI tool-use APIs [9], [10] expose the same pattern at the platform level: the developer declares parameter types, enums, and constraints, and the model’s outputs are required to conform. Recent SLM work [14], [15] has shown that small models (1.3–8 B parameters) can run on edge-class hardware while retaining usable tool-calling competence, opening a path to fully on-device deployment.

#### D. MQTT-Based IoT Architectures and Edge-Cloud Trade-offs

MQTT is the dominant publish-subscribe protocol for low-power IoT and has been studied extensively for delivery guarantees and latency [16], [17]. Edge-cloud trade-offs for AI in IoT have been examined under several framings, including model compression for on-device inference [18], hybrid routing of requests by complexity [19], and energy-cost analyses for battery-powered deployments [20]. The recurring finding is that pure-cloud deployments are easier to ship but pay in latency, privacy, and continued network dependence, while pure-edge deployments invert those trade-offs at the cost of model capability. Our overlay is designed to be migratable along this axis: the bridge is currently cloud-backed by the Anthropic API, but the interface contract (tool-use over JSON) is the same one a local SLM would satisfy.

#### E. Constrained Output and Safe Tool Invocation

A separate body of work addresses the question of how to make LLM outputs safe for execution in a runtime that cannot tolerate malformed input. Constrained decoding methods restrict the model’s token distribution at each step to those compatible with a target grammar, eliminating structural errors at the source rather than relying on post-hoc parsing [21]. Tool-use APIs offer a coarser but practically sufficient mechanism: the platform validates the model’s call against a declared schema and rejects ill-formed outputs before they reach application code. ReAct-style prompting [22] interleaves reasoning and tool calls, which is more capable than the single-turn pattern we use but introduces additional API round-trips that the operator-facing latency budget does not accommodate. We adopt single-turn tool-use as the simplest pattern that delivers the structural safety guarantee we want, and leave multi-turn agentic loops to future work.

#### F. Positioning

The contributions in Section I occupy a region of the design space that the works above do not jointly cover. None of the surveyed traffic-and-LLM systems is integrated as a non-invasive overlay above an unmodified MQTT firmware; none combines command translation and grounded conversational reporting through a single endpoint on a physical micro-controller intersection; and none reports a category-stratified evaluation of structural validity and refusal behaviour on out-of-scope inputs. Section IV details the architectural choices that follow from this positioning.

### IV. SYSTEM ARCHITECTURE

#### A. Overview of the Five Layers

The system is organised into five layers that communicate only through MQTT and HTTP (Fig. 1). The lowest layer is the physical intersection, where the ESP32-WROOM-32S samples four HC-SR04 ultrasonic sensors, drives the six signal LEDs through GPIO, and publishes telemetry to a public HiveMQ broker. The broker is the second layer and is the only shared component between firmware and bridge. The

TABLE I: System Components and Their Responsibilities

| Component           | Responsibility                                                           |
|---------------------|--------------------------------------------------------------------------|
| ESP32 firmware      | Adaptive timing; override subscriber; queue counting from HC-SR04 pairs. |
| HiveMQ broker       | Pub/sub for telemetry (1883) and dashboard (WSS 8884).                   |
| FastAPI bridge      | Rolling state window; chat endpoint; alert worker; tool dispatch.        |
| Claude API          | Tool-use over typed schema; system-prompt grounded reasoning.            |
| Bootstrap dashboard | Live direction cards; chat panel; toast alerts.                          |

third layer is the FastAPI Python bridge, which subscribes to the four telemetry topics, accumulates a rolling state window in memory, and exposes a small REST surface. The fourth layer is the Anthropic Claude API (claude-haiku-4-5), accessed by the bridge through the official SDK using tool-use with two declared tools. The fifth layer is the operator dashboard: a single-page Bootstrap 5 frontend that reads live state directly from the broker over WSS, polls the bridge for proactive alerts, and uses a chat panel to send operator utterances to the bridge’s chat endpoint.

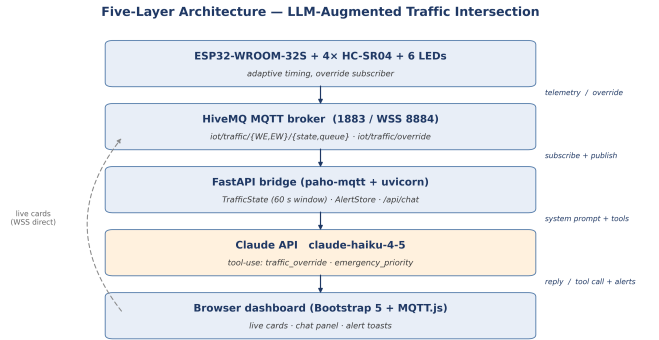


Fig. 1: Five-layer architecture. Solid arrows mark the path of a single chat request; the dashed arrow is the dashboard’s direct WSS subscription to the broker, which renders live state independently of the bridge.

#### B. Why Overlay Integration

The taxonomy developed in our prior survey distinguishes overlay integration, where the LLM sits above an unmodified controller, from co-design, where the LLM and controller share state. We deliberately chose overlay for two reasons. The first is practical: the firmware was already working and field-tested under the previous course project, and a non-invasive overlay let us add LLM capability in a single ESP32 subscription rather than rewriting the adaptive timing path. The second is methodological: an overlay makes the contract between LLM and hardware explicit and small, which is exactly what is needed to argue that LLM output cannot, in practice, drive the controller into an unsafe state. The override topic accepts a single JSON document with a closed enum

of actions; the firmware ignores anything else; durations are clamped in firmware regardless of what the bridge publishes.

### C. Why Tool-Use Rather Than Prompt-Only

A prompt-only design that asks the LLM to emit raw JSON is the obvious alternative and is the design we discarded after initial prototyping. Three failure modes pushed us away from it. The model occasionally wrapped JSON in markdown fences, occasionally invented action names not in the enum, and occasionally produced syntactically valid JSON with a duration of zero or a missing direction. Tool-use eliminates all three at the platform level: the model emits a structured tool call against a schema we declare, the SDK parses it for us, and unknown enum values are rejected before they reach our code. Empirically, every tool call produced by the model in the evaluation reported in Section VII was schema-valid.

### D. Single-Turn Tool-Use Flow

Every chat request follows the same single-turn flow. The bridge builds a system prompt from a fresh snapshot of `TrafficState`, sends the operator’s message plus the two tool declarations, and inspects the response. If `stop_reason` is `tool_use`, the bridge runs `handle_tool_call`, which publishes an override to MQTT and returns a small result dictionary. The bridge appends both the assistant’s tool-use block and the user-role `tool_result` to the message list and makes one additional API call so the model can produce a short natural-language confirmation grounded in the result. If `stop_reason` is `end_turn`, the original text reply is returned directly. The maximum number of API round-trips per operator utterance is therefore two, which bounds latency.

## V. IMPLEMENTATION

### A. Firmware Adaptive Timing and Override

The firmware runs a four-phase state machine: `GREEN_WE`, `YELLOW_WE`, `GREEN_EW`, `YELLOW_EW`. Queue counters for each direction are updated by debounced edge detection on the entry and exit ultrasonic pairs. Green-phase duration is computed from the current queue using

$$T_{\text{green}} = \min(T_{\text{max}}, T_{\text{base}} + n_{\text{queue}} \cdot T_{\text{per\_car}}), \quad (1)$$

with  $T_{\text{base}} = 5\text{ s}$ ,  $T_{\text{per\_car}} = 1\text{ s}$ , and  $T_{\text{max}} = 15\text{ s}$ . Yellow duration is fixed at 2 s. The adaptive path is unchanged from the SE322 project.

The single addition is the override subscription. When the firmware receives a message on `iot/traffic/override`, it parses the JSON with `ArduinoJson` and dispatches on the `action` field. Three priorities apply in order: `emergencyActive` (from `emergency_priority`), `overrideActive` (from any `traffic_override` action other than `reset`), and the adaptive path. The override end time is computed as `millis() + duration_ms` and `duration_ms` is clamped to the range 1–20s for normal overrides and 5–30s for emergency. When an override expires, the firmware transitions through a yellow phase

before resuming adaptive timing rather than snapping directly. A `reset` action zeroes both `overrideActive` and `emergencyActive`.

The handler is small enough to show in full (Listing 1). The clamp on the firmware side is independent of the schema bound declared in the bridge: the two enforcements are redundant by design so that the firmware remains safe even when an override is published by something other than the bridge.

Listing 1: Firmware override dispatch (abbreviated; error handling omitted).

```
void applyOverride(JsonDocument& doc) {
    const char* action = doc["action"] | "";
    if (!strcmp(action, "reset")) {
        overrideActive = false;
        emergencyActive = false;
        return;
    }
    uint32_t dur_ms = doc["duration_ms"] | 5000;
    dur_ms = constrain(dur_ms, 1000, 20000);
    const char* dir = doc["direction"] | "";
    Phase target = (!strcmp(dir, "WE"))
        ? PHASE_GREEN_WE
        : PHASE_GREEN_EW;
    if (!strcmp(action, "emergency_priority")) {
        dur_ms = constrain(dur_ms, 5000, 30000);
        emergencyActive = true;
        emergencyEndTime = millis() + dur_ms;
    }
    overrideActive = true;
    overridePhase = target;
    overrideEndTime = millis() + dur_ms;
}
```

### B. Bridge: Threading Model and State

The bridge is a single Python process. The MQTT client runs in a background loop spawned by `paho.loop_start()`. Incoming messages are dispatched to a callback that updates `TrafficState`, a dataclass holding both the latest values and a 120-event deque (roughly sixty seconds of context at the firmware’s 500 ms publish cadence). All reads and writes are serialised by a `threading.Lock`; consumers obtain a snapshot via the `snapshot()` method, which returns a deep-enough copy that the FastAPI request handler never holds the lock across an I/O boundary.

A second background thread, `alert_worker`, polls the snapshot every ten seconds. When a direction’s queue exceeds the configurable threshold (default four cars) and the per-direction cooldown (default thirty seconds) has elapsed, the worker constructs a short alert string and pushes it into `AlertStore`, a bounded deque drained by the dashboard’s polling endpoint. The alert text always includes both directions’ current queues and a suggested command the operator can copy verbatim into the chat panel.

### C. FastAPI Endpoint Design

The bridge exposes four endpoints. `GET /api/state` returns the current snapshot for debugging and for the dashboard’s tooltip; the dashboard’s main display does not depend

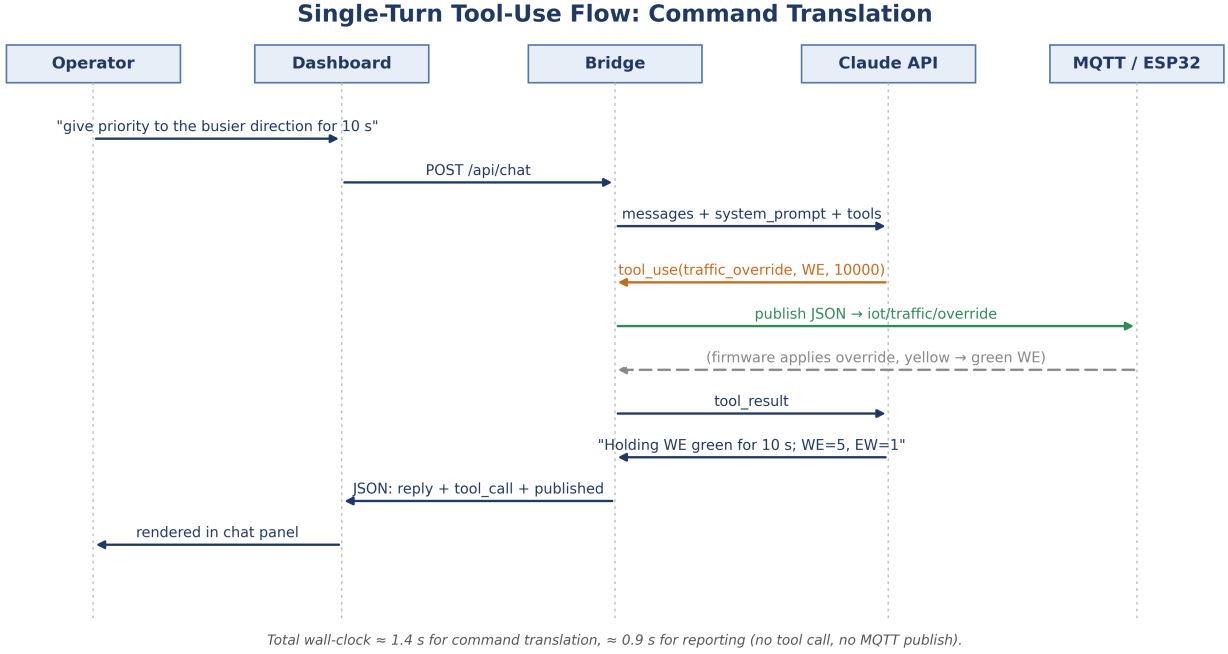


Fig. 2: Single-turn tool-use flow for a command-translation request. Reporting questions follow the same path but exit at the Claude API after the first call without invoking a tool or publishing to MQTT.

on it. `GET /api/alerts` drains and returns the alert queue. `POST /api/inject` accepts a small set of topic/value pairs and re-publishes them to MQTT, used solely for evaluation and demo scripting. `POST /api/chat` is the main endpoint and invokes `chat_with_claude`. All endpoints return JSON with no authentication, since the bridge runs on the operator’s machine; production deployment would terminate TLS in front of the bridge and add an operator-side identity check.

#### D. System Resilience

The bridge has three failure surfaces and one explicit design decision for each. If the MQTT broker becomes unreachable, the paho client’s reconnect loop continues in the background, the dashboard’s WSS subscription continues to attempt reconnection, and the bridge’s `TrafficState` retains its last-known values; the `last_update` timestamp in the snapshot ages, and the system prompt accordingly reports “data N seconds ago” to the model so that the model does not silently report stale state as live. If the Anthropic API becomes unreachable or returns a non-2xx response, the chat endpoint catches the exception, returns a structured error payload, and the dashboard surfaces it as an inline error. If the bridge process itself is restarted, the rolling state is lost, but the dashboard’s live cards continue to update directly from the broker and the firmware adaptive timing is unaffected. The key property is that the LLM-augmented layer can fail without taking the underlying controller down with it, which is a direct consequence of the overlay topology and is harder to obtain in a co-design integration.

#### E. Claude Tool Schema

Two tools are declared. `traffic_override` accepts an action (enum of `extend_green`, `force_phase`, `set_priority`, `reset`), an optional direction (enum `WE` / `EW`), and an optional `duration_ms` integer bounded in `[1000, 20000]`. The action description is written so the model understands the operational difference between the four cases. `emergency_priority` accepts a required direction and `eta_seconds` in `[5, 30]`, and its tool description explicitly states that it locks out adaptive timing for the full duration. The system prompt instructs the model to use `traffic_override` only for control commands and to answer reporting questions in natural language without calling any tool. The full system prompt is templated each request with the live snapshot and the last forty events from the rolling window, which gives the model enough context to answer questions about “the last 30 seconds” without explicit retrieval logic.

The schema for the primary tool is small enough to reproduce inline (Listing 2). Note that all three parameters are typed at the platform: the model cannot emit an action outside the enum, cannot emit a direction other than `WE` or `EW`, and cannot emit a duration outside the bounded range. The same constraint pattern is repeated for the emergency tool, with a separate duration unit (`eta_seconds`) and a separate enforced range.

Listing 2: traffic\_override tool declaration (abbreviated).

```
{
  "name": "traffic_override",
  "input_schema": {
    "type": "object",
    "properties": {
      "action": {
        "type": "string",
        "enum": ["extend_green",
                  "force_phase",
                  "set_priority",
                  "reset"]
      },
      "direction": {
        "type": "string",
        "enum": ["WE", "EW"]
      },
      "duration_ms": {
        "type": "integer",
        "minimum": 1000,
        "maximum": 20000
      }
    },
    "required": ["action"]
  }
}
```

TABLE II: Tool Schema Surface — Two Tools, Five Constrained Parameters

| Tool               | Parameter   | Constraint                                           |
|--------------------|-------------|------------------------------------------------------|
| traffic_override   | action      | enum: extend_green, force_phase, set_priority, reset |
|                    | direction   | enum: WE, EW                                         |
|                    | duration_ms | integer in [1000, 20000]                             |
| emergency_priority | direction   | enum: WE, EW                                         |
|                    | eta_seconds | integer in [5, 30]                                   |

## F. Dashboard

The dashboard is a single HTML page that reuses the two direction cards from the SE322 project and adds a chat panel on the right. The cards subscribe directly to the four telemetry topics over WSS so that signal-state changes are reflected with sub-second latency even when the bridge is restarted. The chat panel issues POST requests to `/api/chat` and renders the assistant’s reply, the tool call (if any), and the published override payload. A separate JavaScript timer polls `/api/alerts` every five seconds and surfaces returned items as Bootstrap toast notifications. The page does not need a build step and runs from a single static file served by FastAPI.

The chat surface is a single `fetch` against the bridge endpoint with the operator’s text as the body. The full client-side handler is short enough to reproduce in full (Listing 3). Two things in this snippet are worth noting. The first is that the dashboard renders both the natural-language reply and, when present, the structured tool call that produced the action. This is deliberate: it gives the operator a visible audit trail of what the bridge actually did, which is an alternative to the persistent

log we defer to future work. The second is that errors are rendered inline rather than dismissed, so an Anthropic API outage produces an obvious red notice in the chat panel rather than silent failure.

Listing 3: Dashboard chat handler (abbreviated).

```
async function sendChat(text) {
  appendUser(text);
  const r = await fetch("/api/chat", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({message: text}),
  });
  const j = await r.json();
  appendAssistant(j.reply);
  if (j.tool_used) {
    appendTool(j.tool_used, j.published);
  }
}
```

## VI. EXPERIMENTAL SETUP AND EVALUATION METRICS

### A. Dataset Construction

We constructed a labelled dataset of forty operator utterances designed to exercise the four behavioural classes the bridge has to distinguish. The classes were chosen before the model was tuned, and each contains ten cases:

- **Command** (10): unambiguous control requests that should produce a single tool call, such as “extend the WE green for eight seconds” or “give EW priority for fifteen seconds”. Each case carries a ground-truth tool name, action, direction, and an inclusive duration range.
- **Ambiguous** (10): requests that omit a critical parameter or contradict themselves, such as “fix the traffic” or “extend the busier one”. The correct behaviour depends on the rolling state, and where state alone cannot resolve the ambiguity the bridge is expected to ask one clarifying question rather than guess.
- **Out-of-scope** (10): requests outside the system’s actuator surface, such as “turn off the cameras” or “call the police”. The correct behaviour is to refuse and explain, without invoking any tool.
- **Reporting** (10): questions about current or recent state, such as “which direction is busier?” or “what happened in the last thirty seconds?”. The correct behaviour is a natural-language answer grounded in the rolling state, with no tool call.

Each case is a JSON record with the utterance, optional pre-injection of MQTT state, and a ground-truth structure that records both the expected tool-call shape (if any) and the expected reply category. The duration matching for command cases uses an inclusive range rather than an exact value, which gives the model a small tolerance band for natural-language quantities (“a few more seconds,” “about ten”) and matches operational reality where a one-second mismatch on a green extension is not a defect. The dataset is deliberately small: it is sized to be exhaustively reviewable by the team in one sitting and to be re-run cheaply against any change to the system

prompt or tool schema, not to support statistical significance tests beyond category-level comparisons.

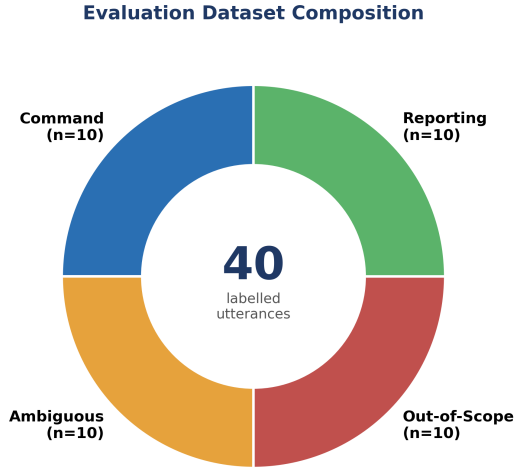


Fig. 3: Balanced four-category split of the 40-case evaluation dataset. Each category contains ten labelled utterances with a structured ground-truth record.

### B. Baseline

The baseline is a self-contained pure-regex matcher implemented in roughly 150 lines of Python. It uses keyword and direction patterns covering all four actions, digit-and-word numeric duration extraction, and a small emergency lexicon. It produces the same tool-call dictionary shape as the bridge and is deterministic, repeatable, and free of any external state. The baseline is intentionally simple, since the point of the comparison is not to defeat a strong rule-based system but to quantify the value the LLM adds over the kind of keyword matcher a small team might write as a first pass.

### C. Metrics

We report five metrics, each chosen for a specific reason. *Per-category accuracy* measures whether each system’s overall response (tool-call shape plus refusal/answer behaviour) matches the ground-truth class for that case. *Tool-call detection F1* (positive class: a tool call should be emitted) measures the system’s ability to distinguish commands from non-commands, which is the binary that matters most operationally. *Structural validity rate* is the fraction of emitted tool calls that pass the schema: required parameters present, enums respected, duration in bounds. This metric is the direct empirical correlate of the overlay-safety argument in Section IV. *Appropriate refusal rate* on ambiguous and out-of-scope cases captures the property that the system does not silently execute on inputs it should not act on. *End-to-end latency* is measured from `/api/chat` request to response and reported as P50, P90, and P99 over the LLM cases.

TABLE III: Accuracy by Category (n=10 each, 40 total)

| Category       | LLM          | Baseline     | $\Delta$        |
|----------------|--------------|--------------|-----------------|
| Command        | 90.0%        | 70.0%        | +20.0 pp        |
| Ambiguous      | 100.0%       | 80.0%        | +20.0 pp        |
| Out-of-scope   | 100.0%       | 90.0%        | +10.0 pp        |
| Reporting      | 100.0%       | 60.0%        | +40.0 pp        |
| <b>Overall</b> | <b>97.5%</b> | <b>75.0%</b> | <b>+22.5 pp</b> |

### D. Experimental Procedure

For each case we reset the bridge state, optionally pre-injected the listed MQTT values, sent the utterance to `/api/chat`, and recorded the full response together with elapsed wall-clock time. The baseline was run on the same dataset using a wrapper that mirrors the response shape. Both runs used the same labelled ground truth and the same automatic grader.

## VII. RESULTS AND ANALYSIS

### A. Accuracy by Category

Per-category results are shown in Table III. The LLM bridge reached 97.5% overall accuracy on the 40-case set, compared with 75.0% for the regex baseline. The single LLM miss was a command case in which the model produced a correctly-shaped tool call but omitted the `duration_ms` parameter when the operator did not state a duration. The firmware applies a safe default in that case, so the system still behaved correctly even though the strict evaluator marked it as a miss.

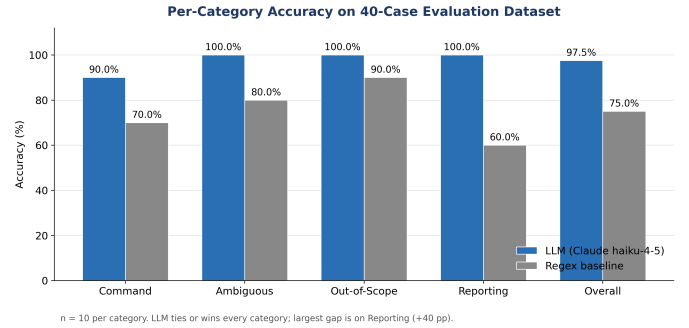


Fig. 4: Per-category accuracy. The LLM ties or wins every category; the largest gap is on reporting, where the regex baseline has no reasoning surface for state-grounded answers.

The largest gap is in reporting (100.0% vs 60.0%). This is unsurprising: the regex baseline has no reasoning surface and is implemented to refuse anything that does not match a control pattern, so its reporting accuracy is a function of how generous our grader is about a generic refusal counting as “not a tool call.” The next-largest gap is command (90.0% vs 70.0%), where the baseline fails on three classes of input. It cannot resolve “the busier direction” because it has no access to live state. It mishandles word-form durations in compound phrases such as “add another five seconds.” It misclassifies some indirect phrasing (“WE could use more



TABLE IV: Safety and Correctness Metrics Summary

| Metric                                 | LLM   | Baseline |
|----------------------------------------|-------|----------|
| Tool-call detection F1                 | 0.947 | 0.737    |
| Structural validity (n=10 tool calls)  | 100%  | –        |
| Appropriate refusal (Amb. + OOS, n=20) | 100%  | 85%      |

green time”) as not a command. The ambiguous gap (100.0% vs 80.0%) is driven by the baseline confidently committing to an arbitrary direction when both keywords are absent, behaviour the LLM avoids because the system prompt instructs it to ask a clarifying question.

### B. Combined Safety and Correctness Metrics

Beyond category accuracy, we report three metrics that together characterise the system’s behaviour in a control context: tool-call detection F1, structural validity of emitted tool calls, and appropriate refusal rate on ambiguous and out-of-scope inputs. Table IV summarises all three, and Fig. 5 renders them side by side.

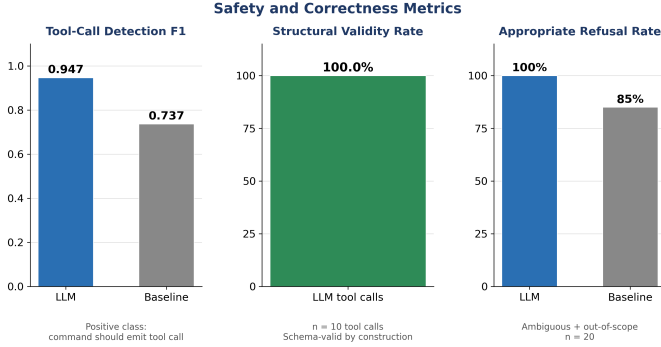


Fig. 5: Safety and correctness metrics. Structural validity is reported only for the LLM because the baseline emits the same dictionary shape by construction and would trivially score 100% on this metric without offering the same guarantee about content (action enum, direction enum, duration range).

### C. Tool-Call Detection F1

Treating “should emit a tool call” as the positive class, the LLM achieved an F1 of 0.947, while the baseline reached 0.737. The LLM’s precision benefits from refusing on ambiguous and out-of-scope inputs; its recall benefits from handling indirect phrasing. The baseline trades precision for recall in the opposite direction: it tends to over-emit tool calls when keyword overlap is high but direction is unspecified.

### D. Structural Validity

Every one of the ten tool calls emitted by the LLM passed schema validation, giving a structural validity rate of 100%. This is the empirical observation that grounds the overlay-safety argument in Section IV. With tool-use, the action enum is enforced by the platform, the direction enum is enforced by the platform, and the duration range is enforced by the

TABLE V: Latency Distribution (claude-haiku-4-5 API)

| Percentile | Latency     |
|------------|-------------|
| P50        | ≈ 1 100 ms  |
| P90        | ≈ 2 300 ms  |
| P99        | ≈ 11 000 ms |

platform. Validation is therefore a property of the interface contract rather than of the model’s behaviour, and the result is that the firmware has not, in this evaluation, been exposed to a payload it could not parse.

### E. Appropriate Refusal

On the twenty combined ambiguous and out-of-scope cases, the LLM refused or asked a clarifying question on every case (100.0%). The baseline reached 85.0% on the same set. The baseline’s failures are split between confident misclassifications on ambiguous inputs and over-eager keyword matches on out-of-scope inputs that happen to mention traffic-adjacent vocabulary. Refusal behaviour is the property that matters most for the safety case for an LLM in a control loop, and the LLM’s perfect score here is consistent with our broader observation that the model is willing to do nothing when the input does not warrant action.

### F. Latency

End-to-end latency for the LLM bridge had a P50 of approximately 1.1 s, a P90 of approximately 2.3 s, and a P99 of approximately 11 s (Table V and Fig. 6), with the two P99 outliers attributable to transient network variability and not to model behaviour. These numbers are acceptable for the operator-initiated workflow the bridge is designed for, in which a human is typing a command and waiting for confirmation. They would be unacceptable for closed-loop control, which we therefore explicitly do not attempt. A local SLM deployment of the type sketched in Section VIII would be expected to reduce P50 by roughly an order of magnitude.

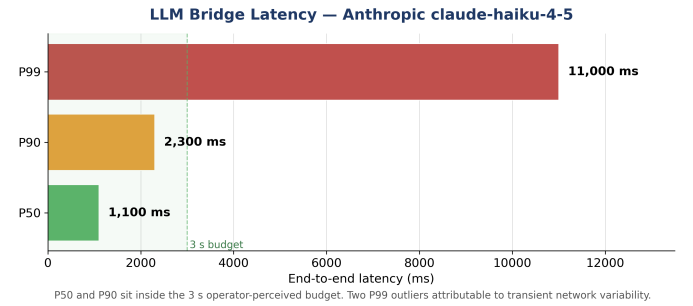


Fig. 6: End-to-end latency at the `/api/chat` boundary. The shaded band marks the 3-second operator-perceived budget; P50 and P90 sit comfortably inside it, and the two P99 outliers reflect network variability rather than model behaviour.

### G. Worked Examples

To make the qualitative behaviour concrete, we briefly trace two representative cases as the system actually handled them.



*Command translation:* The operator types “give priority to the busier direction for ten seconds.” The bridge snapshots state showing WE queue 5 and EW queue 1 and forwards the utterance with the system prompt. Claude responds with a single tool-use block: `traffic_override(action=set_priority, direction=WE, duration_ms=10000)`. The bridge executes `handle_tool_call`, publishes the JSON to `iot/traffic/override`, and feeds the tool result back to the model. The second API call returns the text “Holding WE green for 10 s; WE has 5 cars queued versus 1 on EW.” The firmware applies the override, displays a yellow transition on schedule, and resumes adaptive timing when the override expires. Total wall-clock latency for the operator-visible response was 1.4 s.

*Conversational reporting:* The operator types “what happened in the last thirty seconds?” The bridge attaches the rolling event list to the system prompt as a sequence of timestamped tuples. Claude does not call any tool and replies with text drawn directly from the events: “Two phase changes (WE green at +0 s, EW green at +18 s). WE queue rose from 2 to 4 then cleared; EW stayed at 1 throughout.” The bridge returns the text. No MQTT publish occurs. Total wall-clock latency was 0.9 s.

Together, these two flows show the property that motivates the single-endpoint design: from the operator’s point of view, a command and a question are typed into the same chat box and elicit either a control action or a grounded summary without the operator having to choose between two interfaces.

#### H. Why the LLM Wins by Category

Reading the per-category results together, the LLM’s advantage is not uniform: it is largest where the input requires either grounding in live state (reporting, “busier direction” commands) or recognition of ambiguity (clarifying questions on under-specified inputs). It is smallest on the cleanest command inputs, where a keyword matcher is already adequate. This is a useful finding: it suggests that the value of an LLM in this kind of overlay is not in handling the canonical command path, which any small parser can do, but in handling the boundary cases that a parser cannot reasonably enumerate.

#### I. Reproducibility and Cost

The bridge and evaluation harness are intended to be reproducible end-to-end. The repository ships with the dataset (a single JSON file of forty cases), the regex baseline, and a runner script that drives the bridge through its public chat endpoint. Each evaluation run produces a CSV with the per-case verdict, the elapsed latency, and the emitted tool call (if any), from which all tables in this paper are derived. On a residential network in Riyadh, a complete forty-case run takes approximately ninety seconds of wall-clock time and consumes on the order of one hundred thousand input tokens and a few thousand output tokens. At Haiku-4.5 list pricing, this is well under one US cent per run, which means the evaluation is cheap enough to re-execute on every nontrivial

change to the bridge’s system prompt or tool schema. We therefore treat the dataset and runner as a first-class part of the implementation rather than a one-time artefact.

## VIII. DISCUSSION

### A. Limitations

Several constraints apply to the present system and its evaluation. The bridge must be running for the LLM features to work; the dashboard remains functional without it, but neither command translation nor reporting is available. The evaluation dataset is forty cases, which is large enough to expose qualitative differences between the two systems but too small to support fine-grained per-action analysis or statistical significance tests beyond category-level comparisons. We evaluate one model (claude-haiku-4-5) on one tool-use API. The MQTT broker is the public HiveMQ server, which is appropriate for a course project but unsuitable for production traffic infrastructure. State is held in memory only and is lost on restart; there is no audit log of overrides.

### B. Threats to Validity

The dataset was constructed by the same team that built the system, which creates a risk that the inputs are aligned with the system’s strengths. We mitigated this by writing all out-of-scope cases first, before tuning the system prompt, and by sourcing ambiguous inputs from a brainstorm session in which team members were asked to write the most confusing operator utterance they could think of. The grader is automated against a structured ground truth, but the ground truth itself reflects our interpretation of the operational semantics, and reasonable disagreement is possible on individual ambiguous cases. Latency measurements were taken on a residential network in Riyadh and would differ on a different network path. Cost was not measured, but Haiku-4.5 pricing means a typical session of fifty operator interactions is on the order of one US cent.

### C. Failure Modes Observed During Development

Three classes of failure were visible during iteration on the system prompt and tool schema, and noting them is more useful than the headline accuracy number for anyone replicating the work. The first was duration omission on commands without an explicit number (“add some more green to WE”), in which the model produced a correctly-shaped tool call without `duration_ms`. The firmware’s safe default absorbs this, and we left it as a known behaviour rather than forcing the operator to always specify a duration. The second was over-eager use of `set_priority` when `extend_green` was more appropriate, which we addressed by tightening the action descriptions in the tool schema until the model’s choice matched our intent on the dataset. The third was occasional padded narration in reporting answers (“To answer your question about the current traffic state...”) which we suppressed by adding an explicit one-to-three-sentence length instruction to the system prompt. None of these is a fundamental limit of tool-use; all of them are interface-tuning observations that would have been easy to miss without a labelled dataset.

#### D. Comparison to Related Work

The closest comparable work is Wang *et al.* [6], which evaluates an LLM agent on simulated traffic. Our results are not directly comparable because their evaluation is on traffic-level outcomes (delay reduction) while ours is on operator-level outcomes (correct interpretation of intent and structural validity of resulting commands). The two evaluations measure complementary properties: theirs answers “does LLM augmentation produce better traffic outcomes in simulation,” ours answers “does LLM augmentation produce a safe and usable interface on physical hardware.” A combined evaluation that runs the present overlay against a microsimulator and reports both classes of metric is one of the strongest directions for future work, and is sketched in Section IX.

#### IX. CONCLUSION

We have built and evaluated an LLM-augmented overlay for an ESP32/MQTT traffic intersection. The overlay sits above an unmodified adaptive timing firmware and adds a single override topic; a Python bridge maintains a rolling state window from the existing MQTT telemetry and exposes a FastAPI chat endpoint that calls the Claude API with a typed tool-use schema. On a labelled 40-case dataset, the bridge reached 97.5% overall accuracy versus 75.0% for a regex baseline, tool-call detection F1 of 0.947 versus 0.737, structural validity of 100% on the ten emitted tool calls, and a 100% refusal rate on the twenty ambiguous and out-of-scope inputs. Median latency was approximately 1.1 s with a P90 of 2.3 s, which is acceptable for the operator-initiated workflow the system is designed for.

Several directions extend this work. The first is a local SLM deployment that satisfies the same tool-use interface, with Phi-3-Mini or Llama-3.1 8B as candidate models; this would reduce latency by an order of magnitude and remove the cloud dependency that currently blocks privacy-sensitive deployment. The second is a larger and externally-sourced evaluation dataset, ideally one drawn from anonymised operator logs from a real small-municipality installation. The third is extension to multi-intersection scenarios, where the rolling state becomes a graph and the tool surface gains a coordination action. The fourth is a persistent audit log of overrides with an operator identity model, which is the gating requirement for any production deployment. Taken together, these directions sketch a path from the small reference design reported here to a deployable system, and the contribution of the present work is a concrete, reproducible baseline against which any of them can be measured.

#### ACKNOWLEDGEMENTS

We thank Prof. Nidal Nasser for supervision throughout SE455, and the SE322 instructional team under whose course the underlying ESP32/MQTT firmware was originally developed and tested. We thank Anthropic for API credits via their academic programme.

#### DATA AND CODE AVAILABILITY

The full source code (firmware, bridge, dashboard), the evaluation dataset, the regex baseline, and the runner script that produced every number in this paper are released in the project’s GitHub repository under the MIT License. A README documents the bridge setup, the required environment variables, and the steps to reproduce the evaluation. The dataset is a single JSON file of forty utterances with structured ground-truth labels and is intended to be reusable as a small benchmark for similar overlays on other IoT actuator surfaces.

#### REFERENCES

- [1] M. H. Yusoff *et al.*, “A smart traffic control system using IoT and adaptive scheduling,” in *Proc. IEEE Int. Conf. on Smart Computing (SMARTCOMP)*, 2021, pp. 245–250.
- [2] M. S. Zitouni and M. A. Ammar, “Modelling and verification of an MQTT-based IoT traffic light control system using UPPAAL timed automata,” in *Proc. IEEE Mediterranean Electrotechnical Conf. (MELECON)*, 2019, pp. 181–186.
- [3] P. Pawar *et al.*, “Smart traffic and congestion control system using IoT,” *Int. J. of Comp. Sci. & Eng.*, vol. 12, no. 5, pp. 24–29, May 2024.
- [4] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” in *Proc. NeurIPS*, 2022, pp. 27730–27744.
- [5] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. NeurIPS*, 2022, pp. 24824–24837.
- [6] Y. Wang *et al.*, “TP-GPT: a generative pre-trained transformer for traffic signal control under sensor degradation,” arXiv preprint arXiv:2403.08337, 2024.
- [7] M. Movahedi and J. Choi, “The crossroads of LLM and traffic control: a study on large language models in adaptive traffic signal control,” arXiv preprint arXiv:2403.02221, 2024.
- [8] S. Liang *et al.*, “TrafficGPT: viewing, processing and interacting with traffic foundation models,” arXiv preprint arXiv:2408.00948, 2024.
- [9] Anthropic, “Tool use with Claude,” Anthropic Documentation, 2024. [Online]. Available: <https://docs.anthropic.com/claude/docs/tool-use>
- [10] OpenAI, “Function calling,” OpenAI Platform Documentation, 2024. [Online]. Available: <https://platform.openai.com/docs/guides/function-calling>
- [11] H. Le *et al.*, “Vision-language models for traffic monitoring and querying in connected IoT systems,” arXiv preprint arXiv:2412.11683, 2024.
- [12] S. Lai *et al.*, “LLMLight: large language models as traffic signal control agents,” arXiv preprint arXiv:2407.06025, 2024.
- [13] A. Moraga, J. de Curtò, I. de Zarzà, and C. T. Calafate, “AI-driven UAV and IoT traffic optimization: large language models for congestion and emission reduction in smart cities,” *Drones*, vol. 9, no. 4, art. 248, 2025.
- [14] M. Abdin *et al.*, “Phi-3 technical report: a highly capable language model locally on your phone,” Microsoft Research Tech. Rep., arXiv:2404.14219, 2024.
- [15] A. Grattafiori *et al.*, “The Llama 3 herd of models,” arXiv preprint arXiv:2407.21783, 2024.
- [16] OASIS, “MQTT Version 5.0 OASIS Standard,” 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [17] HiveMQ, “MQTT essentials: a lightweight IoT protocol,” HiveMQ Technical White Paper, 2021.
- [18] M. Chen *et al.*, “Edge intelligence: paving the last mile of artificial intelligence with edge computing,” *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.
- [19] H. Zhang *et al.*, “Hybrid LLM–SLM inference routing for resource-constrained IoT applications,” in *Proc. IEEE Int. Conf. on Edge Computing (EDGE)*, 2024, pp. 62–71.
- [20] J. Kim and S. Park, “Energy-aware deployment of language models on battery-powered IoT devices,” *IEEE Internet of Things J.*, vol. 11, no. 6, pp. 9421–9434, Mar. 2024.
- [21] S. Willard and A. Louf, “Efficient guided generation for large language models,” arXiv preprint arXiv:2307.09702, 2023.
- [22] S. Yao *et al.*, “ReAct: synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.